

PCEP Mock Examination

Certified Entry-Level Python Programmer Practice Test

Exam Syllabus: PCEP-30-02

Total Questions: 30

Duration: 40 Minutes

Passing Score: 70%

Candidate Instructions

- This exam mimics the structure, format, and edge cases seen on the official Python Institute PCEP exam.
- It contains multiple question formats: Single-Choice, Multiple-Choice, Drag & Drop, Fill-in-the-Blank, and Target Order.
- **This version does not contain an answer key.** Treat this as a real test run. Ensure you track answers independently.
- Pay extremely close attention to loops, indexing boundaries, mutability rules, and hidden operations.

Section 1: Computer Programming & Python Fundamentals (18%)

1. Which of the following statements completely and accurately describes the role of the Python interpreter?

- A) It translates the source code entirely into an executable machine code file before initial program execution.
- B) It checks syntax rules during a compilation stage and then executes bytecode line-by-line using a virtual machine environment.
- C) It evaluates logical semantics first, ignoring syntax errors until runtime occurs during execution.
- D) It converts low-level instructions explicitly into high-level human-readable code blocks.

2. What is the value outputted by the following execution sequence?

```
x = 0o11
y = 0x0B
print(x == y)
```

- A) True
- B) False**
- C) SyntaxError
- D) TypeError

3. Select all tokens below that represent valid user-defined variable identifiers in Python. (Select TWO options)

- A) `_lambda`**
- B) `yield`
- C) `__2nd_val`**
- D) `true-flag`
- E) `global.var`

4. What is the precise terminal output of this Python command?

```
print("Alpha", "Beta", sep="-", end="*")
print("Gamma")
```

- A) Alpha-Beta*Gamma**
- B) Alpha Beta*Gamma
- C) Alpha-Beta*
Gamma
- D) Alpha-Beta Gamma

5. Analyze the code segment below. What data type will variable `z` hold at the completion of execution?

```
x = input("Enter standard integer scale: ")
y = 4.0
z = x * int(y)
```

- A) `int`
- B) `float`
- C) `str`
- D) `TypeError` statement

6. Given the mathematical formulation, determine the value stored inside variable `result`:

```
result = 2 * 3 ** 2 // 4 + 1
```

- A) 5
- B) 5.5
- C) 4
- D) 10

7. What is the value resulting from the complex logical expression evaluated below?

```
x = True
y = False
z = False
ans = x or y and not z and y
```

- A) `True`
- B) `False`
- C) `None`
- D) `ValueError`

Section 2: Control Flow – Conditional Blocks and Loops (29%)

8. What is the output of the conditional statement tree shown below?

```
x = 5
if x > 2:
    if x < 4:
        print("A")
    elif x == 5:
        print("B")
else:
    print("C")
```

- A) A
- B) B**
- C) C
- D) Nothing is printed.

9. How many iterations will the execution of the following loop perform?

```
val = 1
while val < 10:
    val *= 2
    if val == 8:
        continue
    val += 1
```

- A) 3**
- B) 4
- C) 5
- D) The loop runs infinitely.

10. What list of numbers will the structural loop code loop compile?

```
for k in range(4, -2, -2):
    print(k, end=" ")
```

- A) 4 2 0**
- B) 4 2 0 -2
- C) 4 1 -2
- D) Empty output string

11. What is the ultimate printed result of this control sequence?

```
s = 0
for i in range(3):
    for j in range(2):
        if i == j:
            break
        s += 1
print(s)
```

- A) 1
- B) 2
- C) 3
- D) 4

12. Review the nested iteration loop block. What value is printed?

```
v = 1
for x in range(2):
    v += x
    for y in range(1):
        if v % 2 == 0:
            continue
        v += y
print(v)
```

- A) 1
- B) 2
- C) 3
- D) 4

13. [Drag and Drop] Match the execution statement to its correct functionality loop description.

break

continue

pass

1. Skips the remainder of the current loop block iteration and evaluates the condition check step directly:

2. Exits the entire loop system structure block instantly, jumping past all loop syntax rules: _____

3. Acts as a null syntax operational placeholder, doing absolutely nothing during functional running:

14. What occurs during the runtime evaluation of the loop block layout defined down below?

```
x = [1, 2]
for i in x:
    x.append(i)
    if len(x) > 4:
        break
print(x)
```

- A) [1, 2, 1, 2]
- B) [1, 2, 1, 2, 1]
- C) RuntimeError: Structured array modified during active loops.
- D) [1, 2]

15. What will be the console output of this conditional expression evaluation loop?

```
a = 10
b = 20
c = 30
if a < b or b > c and c == a:
    print("Condition Alpha Passed")
else:
    print("Condition Beta Passed")
```

- A) Condition Alpha Passed
- B) Condition Beta Passed
- C) NameError
- D) SyntaxError

Section 3: Data Collections – Tuples, Dictionaries, Lists, & Strings (25%)

16. What does this variable slicing technique produce when the terminal prints?

```
lst = [10, 20, 30, 40, 50]
print(lst[-1:-4:-2])
```

- A) [50, 30]
- B) [50, 40, 30]
- C) [20, 40]
- D) []

17. Consider the variable state changes below. What is outputted by the final evaluation step?

```
list_one = [1, 2, 3]
list_two = list_one
list_two[0] = 99
print(list_one)
```

- A) [1, 2, 3]
- B) [99, 2, 3]
- C) [99, [1, 2, 3]]
- D) TypeError: list assignment values error

18. What is the printed result after modifying the multidimensional collection template?

```
matrix = [[1, 2], [3, 4]]
matrix[0].append(matrix[1])
print(len(matrix[0]))
```

- A) 2
- B) 3
- C) 4
- D) AttributeError

19. Analyze this dictionary processing code sequence. What is printed to the standard terminal layout?

```
data = {"a": 1, "b": 2}
val = data.get("c", data.get("b", 3))
print(val)
```

- A) 1
- B) 2
- C) 3
- D) None

20. What is the final value output from this string transformation matrix logic?

```
text = "Python"
text = text.upper()
text[0] = "p"
print(text)
```

- A) pPYTHON**
- B) PYTHON
- C) TypeError
- D) ValueError

21. What value results from using the index validation mechanism shown here?

```
sample = "level"
print(sample.find("e", 2))
```

- A) 1
- B) 3**
- C) -1
- D) ValueError

22. What item value is printed when evaluating the variable declaration set down below?

```
my_tuple = (1, 2, [3, 4])
my_tuple[2][0] = 9
print(my_tuple)
```

- A) (1, 2, [3, 4])
- B) (1, 2, [9, 4])**
- C) TypeError: item assignment within standard tuples handles error
- D) (1, 2, 9)

Section 4: Functions and Exceptions (28%)

23. What value prints from this variable tracking context framework inside the scope function script?

```
def update_val(n):  
    n += [4]  
    return n  
  
base = [1, 2, 3]  
new_list = update_val(base)  
print(base == new_list)
```

- A) True
- B) False**
- C) AttributeError
- D) NameError

24. Analyze the keyword position tracking properties used to execute this function:

```
def compute(x, y=2, z=3):  
    return x * y + z  
  
print(compute(z=1, x=2))
```

- A) 5**
- B) 7
- C) 8
- D) TypeError

25. What happens during compilation and runtime when executing this specific structure schema script?

```
def verify():  
    global global_flag  
    global_flag = "Active"  
  
verify()  
print(global_flag)
```

- A) Active
- B) NameError: global_flag variable not declared at baseline scope**
- C) UnboundLocalError
- D) None

26. What precise block gets logged during execution of the specific handling sequence down below?

```
try:
    val = int("abc")
except ValueError:
    print("Value Catch")
except TypeError:
    print("Type Catch")
else:
    print("Else Block")
```

- A) Value Catch**
- B) Type Catch
- C) Value Catch followed instantly on newline by Else Block
- D) The script crashes with unhandled exception trace logs.

27. Determine what prints out during the variable state verification algorithm loop here:

```
def run_calc(a, b):
    try:
        return a // b
    except ZeroDivisionError:
        return -1
    finally:
        return -2

print(run_calc(4, 2))
```

- A) 2
- B) -1
- C) -2**
- D) No values outputted due to structural conflict crashes.

28. What value is bound inside variable validation state matrix result down below?

```
def process(factor):  
    def factor_calc(val):  
        return val * factor  
    return factor_calc  
  
action = process(3)  
print(action(4))
```

- A) 12**
- B) 7
- C) TypeError
- D) NameError

29. Identify the structural syntax outcome of this data evaluation process:

```
def process_items(items):  
    items.sort()  
  
vals = (3, 1, 2)  
process_items(vals)  
print(vals)
```

- A) (1, 2, 3)
- B) (3, 1, 2)
- C) AttributeError**
- D) TypeError

30. Fill in the blank to complete the expected exception execution path layout block:

```
try:  
    # Segment block checks  
    raise Exception("System Failure")  
except Exception as err:  
    print(_____) # Must print exactly: System Failure
```

- A) err.message
- B) err
- C) err.args[0]**
- D) Both B and C are valid mechanisms to achieve this output string.